SPRING CLOUD MASTERCLASS

Complete Professional Tutorial

Building Resilient Microservices from Discovery to Service Mesh

Eureka	**Gateway**	**Feign**	**Load Balancer**
Config Server	**Resilience4j**	**Consul**	**Service Mesh**

MediCare Hospital Management System — Real-World Project

Spring Boot 3.x | Java 21 | Spring Cloud 2023.x

Introduction

This tutorial is a complete, classroom-style guide to Spring Cloud — the leading framework for building cloud-native microservices in Java. It covers every major component from service registration to service mesh, using a real-world Hospital Management System (MediCare) as the running project throughout.

Who This Tutorial Is For

- Java developers who have Spring Boot experience and want to move into microservices
- Backend engineers preparing for senior or architect-level roles
- Teams migrating from monolithic architectures to distributed systems
- Engineers who want deep conceptual understanding, not just copy-paste configuration

What You Will Build

Throughout this tutorial, you will build the MediCare Hospital Management System — a multi-service application consisting of:

- Doctor Service — manages doctor profiles, schedules, and availability
- Patient Service — handles patient registration and medical records
- Appointment Service — coordinates booking between doctors and patients
- Billing Service — processes payments and generates invoices
- Notification Service — sends SMS/email confirmations
- API Gateway — single entry point, routing, authentication, rate limiting

Technology Stack

Component	Technology	Version
Core Framework	Spring Boot	3.2.x
Service Discovery	Spring Cloud Netflix Eureka / Consul	2023.x
API Gateway	Spring Cloud Gateway	2023.x
HTTP Client	OpenFeign	2023.x
Resilience	Resilience4j	2.x
Configuration	Spring Cloud Config Server	2023.x
Service Mesh	Istio + Envoy	1.20.x
Database	MySQL 8.x	8.0
Build Tool	Apache Maven	3.9.x
Java Version	OpenJDK	21 (LTS)

How To Use This Tutorial

Each module begins with the PROBLEM it solves — understanding the pain point first makes the solution intuitive. Then comes the architecture, the implementation, common pitfalls, and a summary table. Code examples use application.properties (not YAML) for maximum readability.

> **NOTE: All code examples in this tutorial use application.properties format.**
>
> If your project uses application.yml, the property keys are identical — only the format differs.
>
> Example: spring.application.name=doctor-service in properties is equivalent to
>
> spring: application: name: doctor-service in YAML.

MODULE 01

Eureka Server

Service Discovery & Dynamic Registration

Chapter 1: Eureka — Service Discovery

1.1 The Problem — How Do Services Find Each Other?

In a monolith, components find each other via direct method calls. In microservices, services are separate processes running on different machines, different ports, or different containers — and those locations change constantly. Kubernetes reschedules pods. Auto-scaling adds and removes instances. Blue-green deployments swap IPs. How does Appointment Service know where to find Doctor Service right now?

The naive solution — hardcoding IPs — fails immediately in any real deployment:

 # The Hardcoded IP Anti-Pattern

```
# WRONG — hardcoded IP in application.properties
doctor.service.url=http://192.168.1.45:8081

# What happens when:
#  - Doctor Service is restarted? (new port or IP)
#  - You scale to 3 instances? (which IP?)
#  - You deploy to Kubernetes? (pod IP changes every restart)
#  - You move to a different data center?
# Answer: everything breaks. Manually.
```

The solution is Service Discovery: a registry that services register with at startup and query when they need to call another service. Spring Cloud Netflix Eureka is the most widely used service discovery solution in the Spring ecosystem.

1.2 Eureka Architecture

Eureka uses a client-server model with three participants:

- **Eureka Server:** The registry. Receives registrations, maintains the service catalog, responds to queries. Typically runs as a standalone Spring Boot application on port 8761.
- **Eureka Client (Service):** Every microservice includes the Eureka Client library. On startup it registers itself with the server. It sends heartbeats every 30 seconds to confirm it is still alive.
- **Eureka Client (Caller):** When Appointment Service needs to call Doctor Service, it queries the Eureka registry for all healthy instances of doctor-service, then picks one using a load balancing algorithm.

Eureka Architecture

```
│                    EUREKA SERVER :8761                          │
│                                                                 │
│  Service Registry:                                              │
│  ┌──────────────────────────────────────────────────────────┐  │
│  │  doctor-service     → 192.168.1.10:8081  HEALTHY         │  │
│  │  doctor-service     → 192.168.1.11:8081  HEALTHY         │  │
│  │  patient-service    → 192.168.1.12:8082  HEALTHY         │  │
│  │  appointment-service→ 192.168.1.13:8083  HEALTHY         │  │
│  │  billing-service    → 192.168.1.14:8084  DOWN            │  │
│  └──────────────────────────────────────────────────────────┘  │
└─────────────────────────────────────────────────────────────────┘
      ↑ REGISTER + heartbeat every 30s    ↑ QUERY: where is doctor-service?
      │                                   │
┌─────────────┐                     ┌──────────────────┐
│Doctor Svc   │                     │ Appointment Service │
│:8081        │                     │ (load balanced call)│
└─────────────┘                     └──────────────────┘
```

1.3 Setting Up Eureka Server

Step 1 — Maven Dependency

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>

<!-- Spring Cloud BOM (in dependencyManagement) -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-dependencies</artifactId>
  <version>2023.0.0</version>
  <type>pom</type>
  <scope>import</scope>
</dependency>
```

Step 2 — Enable Eureka Server

EurekaServerApplication.java

```
@SpringBootApplication
@EnableEurekaServer   // One annotation activates the entire server
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Step 3 — application.properties (Standalone Server)

application.properties — Eureka Server

```
# Server identity
spring.application.name=eureka-server
server.port=8761

# Standalone mode — server does NOT register itself or fetch from itself
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false

# Registry maintenance settings
```

```
eureka.server.enable-self-preservation=true
# Self-preservation: if >15% of heartbeats are missed, Eureka suspects
# a network partition and stops evicting services. Prevents mass eviction
# when the network hiccups briefly.

eureka.server.eviction-interval-timer-in-ms=5000
# How often (ms) to run eviction of expired leases
```

1.4 Setting Up a Eureka Client (Microservice)

Step 1 — Maven Dependency

pom.xml — Each Microservice

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Step 2 — application.properties (Doctor Service)

application.properties — Doctor Service

```
# ── APPLICATION IDENTITY ──────────────────────────────────────────
spring.application.name=doctor-service
# CRITICAL: This is the logical name services use to find each other.
# Appointment Service calls lb://doctor-service — this name must match.

server.port=8081

# ── EUREKA CLIENT CONFIGURATION ───────────────────────────────────
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
# Where is Eureka Server? Multiple: comma-separated URLs for HA.

eureka.client.fetch-registry=true
# Fetch and cache the registry locally. Required for calling other services.

eureka.client.registry-fetch-interval-seconds=30
# Refresh local cache every 30 seconds.

# ── INSTANCE CONFIGURATION ────────────────────────────────────────
eureka.instance.prefer-ip-address=true
# Register with IP address instead of hostname.
# Essential in Docker/Kubernetes where hostnames are not resolvable.

eureka.instance.lease-renewal-interval-in-seconds=30
# Send heartbeat every 30 seconds.

eureka.instance.lease-expiration-duration-in-seconds=90
# If no heartbeat for 90 seconds, Eureka marks instance as expired.
```

1.5 Self-Preservation Mode — Understanding the Most Confusing Feature

Self-Preservation is Eureka's protection against false positives during network issues. Here is the core logic:

- **Normal operation:** Eureka expects N heartbeats per minute from all registered instances.
- **Network partition scenario:** Network blip causes 40% of heartbeats to be lost. Eureka sees 40% of instances as "not beating."
- **Without self-preservation:** Eureka evicts all 40% of instances. Callers get errors for services that are actually running fine.
- **With self-preservation:** Eureka detects: "I'm missing too many heartbeats — this must be a network issue, not real failures." It stops evicting and keeps serving the existing registry (even if stale).

TIP: Self-Preservation is AP behavior — it prioritizes Availability over Consistency.
During a network partition, Eureka continues serving (possibly stale) data rather than refusing to serve at all. This is the right choice for most web applications but not for systems that require strong consistency (consider Consul for CP behavior).

1.6 High Availability Eureka Cluster

In production, never run a single Eureka Server. Run a cluster of 2-3 peers that replicate their registries with each other:

application.properties — HA Eureka Cluster

```
# ── EUREKA SERVER 1 (runs on host: eureka-1) ──────────────────────
spring.application.name=eureka-server
server.port=8761

# This server registers WITH the peer servers and fetches FROM them
eureka.client.register-with-eureka=true
eureka.client.fetch-registry=true

# Point to the OTHER servers (not itself)
eureka.client.service-url.defaultZone=http://eureka-2:8761/eureka/,http://eureka-
3:8761/eureka/

# Each service points to ALL Eureka servers for resilience
# In Doctor Service application.properties:
eureka.client.service-url.defaultZone=http://eureka-1:8761/eureka/,http://eureka-
2:8761/eureka/,http://eureka-3:8761/eureka/
```

1.7 Common Mistakes

1. **spring.application.name not set:** Service registers as UNKNOWN. All lb:// calls fail with "No instances available for UNKNOWN."
2. **eureka.instance.prefer-ip-address=false in Docker:** Service registers with container hostname (e.g., a3f9b2c1) which is not resolvable from other containers. Always set to true.
3. **Single Eureka Server in production:** Eureka server crash = all new service instances cannot register, all services lose ability to discover others. Run at least 2 peers.
4. **Self-Preservation disabled unnecessarily:** Do not set eureka.server.enable-self-preservation=false in production. Network blips cause mass false evictions.

5. **Expecting instant deregistration:** Lease expiry takes up to 90 seconds. Do not expect a crashed service to disappear from the registry immediately.

1.8 Module Summary

Concept	Key Takeaway
@EnableEurekaServer	Single annotation on main class activates the entire Eureka Server
spring.application.name	The logical service name — all callers use this to discover the service
prefer-ip-address=true	Always set in Docker/Kubernetes — container hostnames are not resolvable
Self-Preservation	Stops eviction during network blips — AP model, prioritizes availability
Heartbeat Interval	30 seconds by default — service must send heartbeat within 90 seconds
HA Cluster	Run 2+ peer Eureka servers — each registers with the others
Local Registry Cache	Clients cache registry locally — survive brief Eureka downtime

MODULE 02

Spring Cloud Gateway

Centralized Routing, Auth, and Rate Limiting

Chapter 2: Spring Cloud Gateway

2.1 The Problem — 50 Services, No Front Door

Without a gateway, every client (mobile app, web browser, external API consumer) must know the address of every service directly. This creates several serious problems:

- Security: Every service is publicly exposed. Authentication must be implemented in every service separately.
- CORS: Every service must configure Cross-Origin Resource Sharing headers independently.
- Rate Limiting: Cannot throttle abusive clients without implementing it in every service.
- Routing Changes: Moving a service to a new port requires updating every client.
- Observability: Request logging is scattered across all services with no unified view.

The solution is an API Gateway: a single entry point that sits in front of all services. Every external request goes through the Gateway, which handles cross-cutting concerns once, centrally, and routes the request to the appropriate backend service.

2.2 Core Concepts — Route, Predicate, Filter

Spring Cloud Gateway is built around three concepts that every developer must internalize:

ROUTE: The fundamental unit — defines WHAT traffic to handle and WHERE to send it.
A route has: an ID, a destination URI (lb://service-name), a set of predicates, and a list of filters.

PREDICATE: The condition that determines IF a request matches this route.
Examples: path matches /api/doctors/**, HTTP method is POST, header X-Version equals 2.

FILTER: The transformation applied to matching requests OR responses.
Examples: Add Authorization header, strip first path segment, add rate limiting, log request.

2.3 Setting Up Spring Cloud Gateway

pom.xml — API Gateway

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<!-- IMPORTANT: Exclude spring-boot-starter-web if present -->
<!-- Gateway uses WebFlux (reactive), not Servlet. They conflict. -->
```

application.properties — API Gateway

```
# ── APPLICATION IDENTITY ──────────────────────────────────────
spring.application.name=api-gateway
server.port=8080

# ── EUREKA INTEGRATION ────────────────────────────────────────
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
eureka.instance.prefer-ip-address=true

# ── ROUTE DEFINITIONS ─────────────────────────────────────────

# Route 1: Doctor Service — all /api/doctors/** requests
spring.cloud.gateway.routes[0].id=doctor-service-route
spring.cloud.gateway.routes[0].uri=lb://doctor-service
# lb:// means: use LoadBalancer to find doctor-service instances via Eureka
spring.cloud.gateway.routes[0].predicates[0]=Path=/api/doctors/**
spring.cloud.gateway.routes[0].filters[0]=StripPrefix=1
# StripPrefix=1: remove /api from the path before forwarding
# /api/doctors/1 → /doctors/1 when it reaches Doctor Service

# Route 2: Patient Service
spring.cloud.gateway.routes[1].id=patient-service-route
spring.cloud.gateway.routes[1].uri=lb://patient-service
spring.cloud.gateway.routes[1].predicates[0]=Path=/api/patients/**
spring.cloud.gateway.routes[1].filters[0]=StripPrefix=1

# Route 3: Appointment Service with authentication
spring.cloud.gateway.routes[2].id=appointment-service-route
spring.cloud.gateway.routes[2].uri=lb://appointment-service
spring.cloud.gateway.routes[2].predicates[0]=Path=/api/appointments/**
spring.cloud.gateway.routes[2].predicates[1]=Header=Authorization, Bearer .*
# Second predicate: only route if Authorization header matches Bearer pattern
spring.cloud.gateway.routes[2].filters[0]=StripPrefix=1

# ── GLOBAL FILTERS ────────────────────────────────────────────
# Apply to ALL routes automatically
spring.cloud.gateway.default-filters[0]=AddRequestHeader=X-Gateway-Source,MediCare-Gateway
spring.cloud.gateway.default-filters[1]=AddResponseHeader=X-Response-
Time,${spring.application.name}

# ── ACTUATOR ──────────────────────────────────────────────────
management.endpoint.gateway.enabled=true
management.endpoints.web.exposure.include=health,gateway,metrics
```

2.4 Custom Authentication Filter

A global pre-filter that validates JWT tokens on every request before they reach any microservice:

JwtAuthenticationFilter.java

```java
@Component
public class JwtAuthenticationFilter implements GlobalFilter, Ordered {

    @Value("${jwt.secret}")
    private String jwtSecret;

    // Paths that do NOT require authentication
    private static final List<String> PUBLIC_PATHS = List.of(
        "/api/auth/login",
        "/api/auth/register",
        "/api/public/**"
    );

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String path = exchange.getRequest().getURI().getPath();

        // Skip authentication for public paths
        boolean isPublic = PUBLIC_PATHS.stream()
            .anyMatch(p -> pathMatcher.match(p, path));

        if (isPublic) {
            return chain.filter(exchange);  // Pass through
        }

        // Extract and validate JWT
        String authHeader = exchange.getRequest()
            .getHeaders().getFirst(HttpHeaders.AUTHORIZATION);

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        try {
            String token = authHeader.substring(7);
            Claims claims = Jwts.parser()
                .setSigningKey(jwtSecret).parseClaimsJws(token).getBody();

            // Add user info to headers for downstream services
            ServerHttpRequest modifiedRequest = exchange.getRequest()
                .mutate()
                .header("X-User-Id",    claims.getSubject())
                .header("X-User-Role",  claims.get("role", String.class))
                .build();

            return chain.filter(exchange.mutate().request(modifiedRequest).build());

        } catch (JwtException e) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
    }

    @Override
    public int getOrder() { return -1; }  // Run FIRST, before all other filters
}
```

2.5 Rate Limiting

Spring Cloud Gateway includes Redis-backed rate limiting that prevents API abuse:

application.properties — Rate Limiting

```
# Rate limiting with Redis (add spring-boot-starter-data-redis-reactive to pom)

# Doctor Service route with rate limiting
spring.cloud.gateway.routes[0].id=doctor-service-route
spring.cloud.gateway.routes[0].uri=lb://doctor-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/api/doctors/**
spring.cloud.gateway.routes[0].filters[0]=StripPrefix=1
spring.cloud.gateway.routes[0].filters[1]=name=RequestRateLimiter
spring.cloud.gateway.routes[0].filters[1].args.redis-rate-limiter.replenishRate=10
# replenishRate: tokens added to bucket per second = 10 requests/second
spring.cloud.gateway.routes[0].filters[1].args.redis-rate-limiter.burstCapacity=20
# burstCapacity: max tokens at once = allow bursts up to 20
spring.cloud.gateway.routes[0].filters[1].args.key-resolver=#{@userKeyResolver}
# Rate limit PER USER (not per IP) using a custom KeyResolver bean

# Redis connection
spring.data.redis.host=localhost
spring.data.redis.port=6379
```

2.6 Common Mistakes

6. **Missing WebFlux / importing Web:** Adding spring-boot-starter-web to the Gateway pom.xml causes a startup conflict with WebFlux. Remove it. Gateway uses reactive programming exclusively.
7. **Forgetting StripPrefix:** /api/doctors/1 is forwarded to Doctor Service as /api/doctors/1. Doctor Service has @GetMapping("/doctors/{id}") — the extra /api causes a 404. Always add StripPrefix=1.
8. **Using http:// instead of lb:// in URI:** lb://doctor-service uses Eureka for load-balanced discovery. http://doctor-service:8081 hardcodes a single instance and bypasses load balancing entirely.
9. **Applying Auth filter to health endpoints:** Monitoring systems need to call /actuator/health without a JWT. Explicitly whitelist health check paths in the authentication filter.

2.7 Module Summary

Concept	Key Takeaway
Route = ID + URI + Predicate + Filter	The fundamental building block of the Gateway
lb:// URI prefix	Use Eureka/Consul to load-balance across service instances
Predicate	The condition — path pattern, HTTP method, headers, query params
Filter (pre/post)	Transform requests before sending or responses before returning
StripPrefix=1	Remove the /api prefix segment before forwarding to microservice
GlobalFilter	Apply to ALL routes — ideal for auth, logging, metrics
WebFlux conflict	NEVER import spring-boot-starter-web in Gateway pom — fatal conflict
Rate Limiting	Redis-backed token bucket — replenishRate/burstCapacity per key

MODULE 03

Feign Client

Declarative HTTP Client for Microservices

Chapter 3: OpenFeign — Declarative HTTP Client

3.1 The Problem — Boilerplate HTTP Code

Without Feign, calling another microservice requires writing substantial boilerplate code for every endpoint you consume. Here is what calling Doctor Service from Appointment Service looks like with plain RestTemplate versus Feign:

RestTemplate vs Feign Comparison

```
// WITHOUT FEIGN — RestTemplate boilerplate (17 lines per call)
@Service
public class AppointmentService {

    @Autowired
    @LoadBalanced
    private RestTemplate restTemplate;

    public DoctorDTO getDoctorById(Long id) {
        String url = "http://doctor-service/doctors/" + id;
        try {
            ResponseEntity<DoctorDTO> response =
                restTemplate.getForEntity(url, DoctorDTO.class);
            if (response.getStatusCode().is2xxSuccessful()) {
                return response.getBody();
            }
            throw new RuntimeException("Doctor not found: " + id);
        } catch (RestClientException e) {
            throw new ServiceUnavailableException("Doctor Service unavailable");
        }
    }
}

// WITH FEIGN — clean interface (5 lines total)
@FeignClient(name = "doctor-service")
public interface DoctorClient {
    @GetMapping("/doctors/{id}")
    DoctorDTO getDoctorById(@PathVariable("id") Long id);
}
```

3.2 Setting Up OpenFeign

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

AppointmentServiceApplication.java

```
@SpringBootApplication
@EnableFeignClients   // Scans for @FeignClient interfaces in this package
public class AppointmentServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(AppointmentServiceApplication.class, args);
    }
}
```

3.3 Complete Feign Client — MediCare Example

DoctorClient.java

```
// DoctorClient.java — in Appointment Service
@FeignClient(
    name = "doctor-service",           // Must match spring.application.name of target
    fallbackFactory = DoctorClientFallbackFactory.class  // For resilience
)
public interface DoctorClient {

    // GET /doctors/{id}
    @GetMapping("/doctors/{id}")
    DoctorDTO getDoctorById(@PathVariable("id") Long id);
    // CRITICAL: @PathVariable MUST have explicit name = "id"
    // Without it: "Feign requires name on @PathVariable" exception at startup

    // GET /doctors with query params
    @GetMapping("/doctors")
    List<DoctorDTO> getDoctorsBySpecialization(
        @RequestParam("specialization") String specialization,
        @RequestParam(value = "available", defaultValue = "true") boolean available
    );

    // POST /doctors/{id}/block-slot
    @PostMapping("/doctors/{id}/block-slot")
    void blockTimeSlot(
        @PathVariable("id") Long doctorId,
        @RequestBody TimeSlotDTO slot    // DTO serialized as JSON automatically
    );

    // GET with request headers
    @GetMapping("/doctors/{id}/schedule")
    ScheduleDTO getDoctorSchedule(
        @PathVariable("id") Long doctorId,
        @RequestHeader("X-Date-Range") String dateRange
    );
}
```

3.4 Fallback with FallbackFactory

FallbackFactory is preferred over a simple @Component fallback because it receives the exception that caused the failure — allowing different fallbacks for different failures:

DoctorClientFallbackFactory.java

```
@Component
public class DoctorClientFallbackFactory
        implements FallbackFactory<DoctorClient> {

    private static final Logger log =
        LoggerFactory.getLogger(DoctorClientFallbackFactory.class);
```

```
    @Override
    public DoctorClient create(Throwable cause) {
        log.error("DoctorClient fallback triggered. Cause: {}", cause.getMessage());

        return new DoctorClient() {

            @Override
            public DoctorDTO getDoctorById(Long id) {
                // Return a degraded but meaningful response
                return DoctorDTO.builder()
                    .id(id)
                    .name("Temporarily unavailable")
                    .available(false)
                    .message("Doctor details unavailable. Call reception: 1800-MEDICARE")
                    .build();
            }

            @Override
            public List<DoctorDTO> getDoctorsBySpecialization(
                    String spec, boolean available) {
                // Return empty list — UI shows "No doctors available"
                return Collections.emptyList();
            }

            @Override
            public void blockTimeSlot(Long doctorId, TimeSlotDTO slot) {
                // Cannot block slot — log for manual retry
                log.error("SLOT BLOCK FAILED for doctor {}. Slot: {}. Manual retry
needed.",
                    doctorId, slot);
            }

            @Override
            public ScheduleDTO getDoctorSchedule(Long id, String dateRange) {
                return ScheduleDTO.empty();
            }
        };
    }
}
```

3.5 Feign Configuration — application.properties

application.properties — Feign Client

```
# ── FEIGN GLOBAL DEFAULTS ──────────────────────────────────────────
spring.cloud.openfeign.client.config.default.connect-timeout=2000
# Connection timeout: 2 seconds to establish TCP connection

spring.cloud.openfeign.client.config.default.read-timeout=5000
# Read timeout: 5 seconds to receive response after connection

spring.cloud.openfeign.client.config.default.logger-level=FULL
# FULL logging (for development): logs headers, body, response
# Options: NONE, BASIC, HEADERS, FULL

# ── PER-CLIENT OVERRIDES ───────────────────────────────────────────
# Billing service gets longer timeout (payment processing is slow)
spring.cloud.openfeign.client.config.billing-service.read-timeout=15000

# ── CIRCUIT BREAKER INTEGRATION ────────────────────────────────────
spring.cloud.openfeign.circuitbreaker.enabled=true
# Enables Resilience4j circuit breaker wrapping of all Feign calls.
# Required for fallbackFactory to work.
```

```
# ── COMPRESSION ──────────────────────────────────────────────────
spring.cloud.openfeign.compression.request.enabled=true
spring.cloud.openfeign.compression.response.enabled=true
spring.cloud.openfeign.compression.request.mime-types=application/json
spring.cloud.openfeign.compression.request.min-request-size=2048
# Compress requests larger than 2KB — reduces network overhead
```

3.6 Common Mistakes

10. **@PathVariable without name:** In Feign interfaces, @PathVariable MUST have an explicit name: @PathVariable("id"). Without it, Feign throws an exception at startup: "@PathVariable requires a name."
11. **@RequestBody on GET:** HTTP GET requests cannot have a body. If you need to send complex query criteria, use POST or convert parameters to @RequestParam.
12. **Fallback without spring.cloud.openfeign.circuitbreaker.enabled=true:** Without this property, fallbackFactory is silently ignored. No error — just no fallback when calls fail.
13. **@FeignClient name not matching spring.application.name:** name="DoctorService" when the service registers as "doctor-service" causes "No instances available for DoctorService" — case-sensitive, exact match required.

3.7 Module Summary

Concept	Key Takeaway
@FeignClient(name=...)	name must EXACTLY match spring.application.name of the target service
@PathVariable("id")	Explicit name is REQUIRED in Feign interfaces — not optional
FallbackFactory	Preferred over @Component fallback — receives the causing Throwable
circuitbreaker.enabled=true	Required for fallbackFactory to activate on failures
connect-timeout	Time to establish TCP connection — 2000ms recommended
read-timeout	Time to wait for response — set per-service based on SLA
logger-level=FULL	Use FULL in development, BASIC in production

MODULE 04

Load Balancing

Client-Side, Health-Aware Traffic Distribution

Chapter 4: Spring Cloud LoadBalancer

4.1 Server-Side vs Client-Side Load Balancing

Traditional load balancing is server-side: a dedicated load balancer (HAProxy, AWS ELB, Nginx) sits in front of services and distributes traffic. In Spring Cloud microservices, the load balancing happens client-side — the calling service itself decides which instance to call:

Aspect	Server-Side LB	Client-Side LB (Spring Cloud)
Location	Separate infrastructure component	Inside the calling service (JAR)
Discovery	Configured manually	Fetched from Eureka/Consul
Network Hops	Request → LB → Service	Request → Service (direct)
Health Awareness	Separate health checks	Uses Eureka health status
Algorithms	Round Robin, Least Conn	Round Robin, Random, custom
Setup Complexity	High (infrastructure)	Low (dependency + property)

4.2 Spring Cloud LoadBalancer Setup

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-loadbalancer</artifactId>
</dependency>

<!-- If using RestTemplate instead of Feign: -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

LoadBalancerConfig.java

```
// If using RestTemplate: mark it @LoadBalanced for automatic instance resolution
@Configuration
public class LoadBalancerConfig {

    @Bean
    @LoadBalanced
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder.build();
        // @LoadBalanced intercepts calls to http://service-name/...
```

```
        // and resolves service-name via Eureka automatically
    }
}

// Usage — service name resolves to a real IP via Eureka + LoadBalancer
DoctorDTO doctor = restTemplate.getForObject(
    "http://doctor-service/doctors/42",  // doctor-service resolved via Eureka
    DoctorDTO.class
);
```

4.3 application.properties — LoadBalancer

application.properties — LoadBalancer

```
# ── LOAD BALANCER CONFIGURATION ──────────────────────────────

# Default algorithm: Round Robin (no configuration needed)
# To switch to Random:
spring.cloud.loadbalancer.configurations=random

# ── HEALTH CHECK AWARE LOAD BALANCING ────────────────────────
# Only route to instances that pass their health check
spring.cloud.loadbalancer.health-check.enabled=true
spring.cloud.loadbalancer.health-check.initial-delay=0
spring.cloud.loadbalancer.health-check.interval=25s
# Check each instance health every 25 seconds

spring.cloud.loadbalancer.health-check.path.default=/actuator/health
# Path to call on each instance — must return {"status":"UP"}

# ── ZONE-AWARE ROUTING (Advanced) ────────────────────────────
# Prefer instances in the same availability zone (reduces cross-zone latency)
spring.cloud.loadbalancer.zone=us-east-1a
# Prefer instances with matching metadata.zone — fall back to any zone if none

# ── RETRY CONFIGURATION ──────────────────────────────────────
spring.cloud.loadbalancer.retry.enabled=true
spring.cloud.loadbalancer.retry.max-retries-on-same-service-instance=0
# 0 = Do NOT retry on the same instance that failed (might be truly down)
spring.cloud.loadbalancer.retry.max-retries-on-next-service-instance=2
# Retry up to 2 times on DIFFERENT instances
spring.cloud.loadbalancer.retry.retry-on-all-operations=false
# Only retry on GET by default — NEVER retry POST (could cause duplicates)
```

4.4 Custom Load Balancing Algorithm

VersionAwareLoadBalancerConfig.java

```
// Custom: always prefer instances of the same version (canary-aware routing)
@Configuration
public class VersionAwareLoadBalancerConfig {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> versionAwareLoadBalancer(
            Environment env,
            LoadBalancerClientFactory factory) {

        String serviceName = env.getProperty(LoadBalancerClientFactory.PROPERTY_NAME);
        String preferredVersion = env.getProperty("app.preferred-version", "stable");

        return new ReactorServiceInstanceLoadBalancer() {
```

```
            @Override
            public Mono<Response<ServiceInstance>> choose(Request request) {
                return factory.getLazyProvider(serviceName,
ServiceInstanceListSupplier.class)
                    .getIfAvailable(NoopServiceInstanceListSupplier::new)
                    .get()
                    .next()
                    .map(instances -> {
                        // First try: instances matching preferred version
                        List<ServiceInstance> preferred = instances.stream()
                            .filter(i -> preferredVersion.equals(
                                i.getMetadata().get("version")))
                            .collect(toList());

                        // Fallback: any available instance
                        List<ServiceInstance> pool =
                            preferred.isEmpty() ? instances : preferred;

                        ServiceInstance chosen = pool.get(
                            ThreadLocalRandom.current().nextInt(pool.size()));
                        return new DefaultResponse(chosen);
                    });
            }
        };
    }
}
```

4.5 Common Mistakes

14. **Retrying POST requests:** POST requests create resources. Retrying a failed POST can create duplicate appointments, duplicate payments, duplicate records. Configure retry-on-all-operations=false and only retry GET.
15. **Not enabling health-check aware LB:** Without health checks, the LoadBalancer round-robins to ALL registered instances — including ones that are currently unhealthy. Always enable health-check.enabled=true in production.
16. **@LoadBalanced RestTemplate vs Feign:** If using Feign (recommended), you do NOT also need a @LoadBalanced RestTemplate — Feign integrates with Spring Cloud LoadBalancer automatically via the lb:// Feign client name resolution.

4.6 Module Summary

Concept	Key Takeaway
lb:// prefix	Activates LoadBalancer — resolves service name to instance via Eureka
@LoadBalanced	Marks RestTemplate to intercept service-name URLs for LB resolution
Health-aware LB	Only routes to instances returning {"status":"UP"} at /actuator/health
Round Robin (default)	Distributes requests evenly across all healthy instances
retry-on-all-operations=false	Only retry GET — never retry POST (prevents duplicate records)
Zone-aware routing	Prefer same-zone instances — reduces cross-AZ latency and cost

MODULE 05

Config Server

Centralized Git-Backed Configuration Management

Chapter 5: Spring Cloud Config Server

5.1 The Problem — Configuration Chaos

MediCare has 50 microservices running across 3 environments (dev, staging, prod). That is 150 configuration files to maintain. Every time the database password rotates (every 90 days per security policy), 50 files must be updated and 50 services redeployed. One team manages these files differently from another. A developer checks out a service but cannot find the production database password. A security audit asks: "where are all the places our API key is stored?" The answer is: "in 50 Git repositories."

Spring Cloud Config Server solves this with one Git repository as the single source of truth for all configuration, across all services, all environments.

5.2 Architecture

Config Server Architecture

```
┌──────────────────────────────────────────────────────────────┐
│              GIT REPOSITORY (github.com/medicare/config)      │
│                                                               │
│  application.properties           ← Shared by ALL services    │
│  application-prod.properties      ← Shared, production profile │
│  doctor-service.properties        ← Doctor Service, all envs  │
│  doctor-service-dev.properties    ← Doctor Service, dev only  │
│  doctor-service-prod.properties   ← Doctor Service, prod only │
│  patient-service.properties                                   │
│  appointment-service-prod.properties                          │
└──────────────────────────────────────────────────────────────┘
         │ reads from (clone + pull)
         ▼
┌──────────────────────────────────────────────────────────────┐
│            SPRING CLOUD CONFIG SERVER  :8888                  │
│  Serves config as REST:  GET /{app}/{profile}[/{label}]       │
│  GET /doctor-service/prod → merged config for doctor+prod     │
└──────────────────────────────────────────────────────────────┘
         │ fetches on startup
         ▼
┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│Doctor Svc    │   │Patient Svc   │   │Appointment   │
│(Config Clnt) │   │(Config Clnt) │   │(Config Clnt) │
└──────────────┘   └──────────────┘   └──────────────┘
```

5.3 Setting Up Config Server

application.properties — Config Server

```
# application.properties — CONFIG SERVER
spring.application.name=config-server
server.port=8888

# ── GIT BACKEND ──────────────────────────────────────────────
spring.cloud.config.server.git.uri=https://github.com/medicare-org/medicare-config
spring.cloud.config.server.git.default-label=main
spring.cloud.config.server.git.basedir=/tmp/config-repo
# Local directory where Config Server clones the repo

spring.cloud.config.server.git.username=${GIT_USERNAME}
spring.cloud.config.server.git.password=${GIT_TOKEN}
# Store credentials as environment variables, never in the file

spring.cloud.config.server.git.refresh-rate=30
# Pull fresh changes from Git every 30 seconds

# ── EUREKA REGISTRATION ──────────────────────────────────────
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

# ── ACTUATOR ─────────────────────────────────────────────────
management.endpoints.web.exposure.include=health,info,refresh,bus-refresh,env
```

ConfigServerApplication.java

```
@SpringBootApplication
@EnableConfigServer   // One annotation activates the entire Config Server
@EnableDiscoveryClient
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}
```

5.4 Property Override Priority

When a service requests configuration, Config Server merges properties from multiple files. The most specific file wins:

Priority	File	Applies When
1 (Highest)	doctor-service-prod.properties	doctor-service + prod profile
2	doctor-service.properties	doctor-service, any profile
3	application-prod.properties	any service + prod profile
4 (Lowest)	application.properties	any service, any profile

5.5 Setting Up a Config Client

application.properties — Config Client (Doctor Service)

```
# application.properties — Doctor Service (Config Client)

# ── BASIC IDENTITY (local fallback) ──────────────────────────
spring.application.name=doctor-service
server.port=8081
```

```
# ── CONFIG SERVER LOCATION ──────────────────────────────────────
# Option A: Direct URL (simple, hardcoded)
spring.config.import=configserver:http://config-server:8888

# Option B: Discovery-first (find Config Server via Eureka)
spring.cloud.config.discovery.enabled=true
spring.cloud.config.discovery.service-id=config-server

# ── ACTIVE PROFILE ──────────────────────────────────────────────
# Set via environment variable in production:
# SPRING_PROFILES_ACTIVE=prod
spring.profiles.active=${SPRING_PROFILES_ACTIVE:dev}

# ── FAIL BEHAVIOR ───────────────────────────────────────────────
spring.cloud.config.fail-fast=true
# true = REFUSE to start if Config Server is unreachable.
# false = Start with local properties only (risky in production)

# ── RETRY ───────────────────────────────────────────────────────
spring.cloud.config.retry.initial-interval=1000
spring.cloud.config.retry.max-attempts=6
spring.cloud.config.retry.max-interval=2000
spring.cloud.config.retry.multiplier=1.1
```

5.6 Zero-Downtime Config Refresh

When a configuration value changes in the Git repository, services can pick it up WITHOUT restarting using @RefreshScope and Spring Cloud Bus:

DoctorFeatureConfig.java + Refresh Trigger

```
// Mark beans whose properties should refresh without restart
@Configuration
@RefreshScope  // This bean is destroyed and recreated on refresh
public class DoctorFeatureConfig {

    @Value("${feature.max-appointments-per-day:20}")
    private int maxAppointmentsPerDay;

    @Value("${feature.online-consultation:true}")
    private boolean onlineConsultationEnabled;

    public int getMaxAppointmentsPerDay() { return maxAppointmentsPerDay; }
    public boolean isOnlineConsultationEnabled() { return onlineConsultationEnabled; }
}

// Trigger refresh on ONE instance (manual):
// POST http://doctor-service:8081/actuator/refresh

// Trigger refresh on ALL instances via Spring Cloud Bus:
// POST http://config-server:8888/actuator/bus-refresh
// → Config Server publishes event to RabbitMQ
// → ALL service instances receive event and refresh automatically
```

5.7 Encrypting Sensitive Values

Encryption Configuration

```
# In the Git config repository — doctor-service-prod.properties
# WRONG (plain text in Git):
# spring.datasource.password=Sup3rS3cr3tP@ss

# CORRECT (encrypted with {cipher} prefix)
spring.datasource.password={cipher}AQB3mX9pK2fNR7hW8jT4vL9sX2yC5mQa...

# Config Server decrypts this before serving to services
# Services receive plain text in memory — never written to disk

# ── CONFIG SERVER — Encryption key ──────────────────────────────
# application.properties of Config Server (NOT in Git)
encrypt.key=${ENCRYPT_KEY}
# Set ENCRYPT_KEY as environment variable on the Config Server machine

# To encrypt a new value:
# POST http://config-server:8888/encrypt  (body: your-plain-value)
# Response: AQB3mX9pK2fNR7hW8j...  (paste this as {cipher}...)
```

5.8 Common Mistakes

17. **Config Server fetching its own config:** A circular dependency where Config Server tries to get its own configuration from itself. Config Server's application.properties must be SELF-CONTAINED on the local filesystem, not fetched from any Config Server.
18. **Forgetting @RefreshScope:** You push a config change, call /actuator/refresh, the response shows which properties changed — but your beans still have the old values. Without @RefreshScope, the bean is created once at startup and never recreated. The @Value field appears to update internally but the bean's state does not actually change until it is recreated.
19. **fail-fast=false in production:** The service starts with local default properties when Config Server is unavailable. In production, this means starting with development database credentials. Always use fail-fast=true.
20. **Single Config Server instance:** Config Server itself becomes a critical single point of failure. Run at least 2 instances. Clients cache their configuration and survive Config Server downtime — but new instances cannot start until Config Server recovers.

5.9 Module Summary

Concept	Key Takeaway
Git repository	The heart of Config Server — every change is a commit with full audit trail
File naming	{service}-{profile}.properties — most specific file wins on conflict
spring.config.import	New Spring Boot 3.x way to connect a service to Config Server
fail-fast=true	Refuse to start without Config Server — prevents running with stale config
@RefreshScope	Required on beans whose @Value fields must update without restart
/actuator/bus-refresh	Broadcasts refresh to ALL running instances via Spring Cloud Bus + RabbitMQ

{cipher} prefix	Encrypted values — Config Server decrypts before serving. Never plain passwords in Git.

MODULE 06

Resilience4j

Circuit Breaker, Retry, Bulkhead — Stop Cascading Failures

Chapter 6: Spring Cloud Circuit Breaker — Resilience4j

6.1 The Problem — Cascading Failures

One of the most dangerous failure modes in microservices is the cascading failure: one slow service causes the entire system to collapse. Here is exactly how it happens in MediCare:

Cascading Failure Timeline

```
T=09:15:32  Doctor Service DB becomes slow (30s query lock)
T=09:15:35  Appointment Service calls getDoctorById() — Thread 1: BLOCKED
T=09:15:36  Another booking attempt — Thread 2: BLOCKED
T=09:15:42  All 50 Appointment Service threads: BLOCKED, waiting for timeouts
T=09:15:43  Appointment Service: UNRESPONSIVE — new requests queue up
T=09:15:50  API Gateway calls to Appointment Service: TIMING OUT
T=09:15:55  Patient Service calls Appointment Service: FAILING
T=09:16:00  ENTIRE MEDICARE SYSTEM: DOWN

Root cause: Doctor Service DB was SLOW, not even DOWN.
Result: Complete system failure.
Recovery: Manual restarts, 25 minutes downtime.
```

The Circuit Breaker pattern prevents this by detecting excessive failures early and stopping calls to the failing service — allowing it to recover while other services continue operating normally.

6.2 Circuit Breaker States

State	Behavior	When It Occurs
CLOSED	All calls flow through normally. CB monitors success/failure rates.	Normal operation
OPEN	ALL calls are REJECTED instantly (0ms). Fallback executes. Failing service not called at all.	Failure rate exceeds threshold
HALF-OPEN	LIMITED probe calls sent to test if service recovered. If probes succeed → CLOSED. If fail → OPEN.	After wait duration expires

6.3 Resilience4j Dependencies

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- NOTE: Do NOT use Netflix Hystrix — it is DEPRECATED since 2018.
     Resilience4j is the official replacement. -->
```

6.4 Configuration — application.properties

application.properties — Resilience4j

```
# ── CIRCUIT BREAKER ──────────────────────────────────────────────
resilience4j.circuitbreaker.instances.doctorServiceCB.sliding-window-type=COUNT_BASED
# COUNT_BASED: analyze last N calls
# TIME_BASED: analyze calls in last N seconds

resilience4j.circuitbreaker.instances.doctorServiceCB.sliding-window-size=10
# Analyze the last 10 calls

resilience4j.circuitbreaker.instances.doctorServiceCB.failure-rate-threshold=50
# Open circuit if 50%+ of last 10 calls failed (= 5 or more failures)

resilience4j.circuitbreaker.instances.doctorServiceCB.minimum-number-of-calls=5
# Do not open circuit until at least 5 calls have been made.
# Prevents opening on the very first failure.

resilience4j.circuitbreaker.instances.doctorServiceCB.wait-duration-in-open-state=30s
# Stay OPEN for 30 seconds before going HALF-OPEN

resilience4j.circuitbreaker.instances.doctorServiceCB.permitted-number-of-calls-in-half-
open-state=3
# Send 3 probe calls in HALF-OPEN. All must succeed to CLOSE.

resilience4j.circuitbreaker.instances.doctorServiceCB.automatic-transition-from-open-to-
half-open-enabled=true
# Automatically transition after wait-duration. Do not require manual trigger.

resilience4j.circuitbreaker.instances.doctorServiceCB.record-
exceptions[0]=java.io.IOException
resilience4j.circuitbreaker.instances.doctorServiceCB.record-
exceptions[1]=java.util.concurrent.TimeoutException
# Only these exceptions count as failures for the circuit breaker.

resilience4j.circuitbreaker.instances.doctorServiceCB.ignore-
exceptions[0]=com.medicare.exceptions.DoctorNotFoundException
# Business exceptions (404 Not Found) do NOT open the circuit.
# A missing doctor is a valid business scenario, not a system failure.

# ── RETRY ────────────────────────────────────────────────────────
resilience4j.retry.instances.doctorServiceRetry.max-attempts=3
resilience4j.retry.instances.doctorServiceRetry.wait-duration=500ms
resilience4j.retry.instances.doctorServiceRetry.enable-exponential-backoff=true
resilience4j.retry.instances.doctorServiceRetry.exponential-backoff-multiplier=2
# Retry delays: 500ms → 1000ms → 2000ms (exponential backoff)
```

```
# ── TIME LIMITER ──────────────────────────────────────────────────
resilience4j.timelimiter.instances.doctorServiceTL.timeout-duration=3s
# Any call to Doctor Service that takes > 3s is cancelled and treated as failure.

# ── BULKHEAD ──────────────────────────────────────────────────────
resilience4j.bulkhead.instances.doctorServiceBH.max-concurrent-calls=20
# Max 20 concurrent calls to Doctor Service. Beyond this: BulkheadFullException.
resilience4j.bulkhead.instances.doctorServiceBH.max-wait-duration=100ms

# ── MONITORING ────────────────────────────────────────────────────
resilience4j.circuitbreaker.instances.doctorServiceCB.register-health-indicator=true
management.health.circuitbreakers.enabled=true
management.endpoints.web.exposure.include=health,circuitbreakers,circuitbreakerevents
```

6.5 Using Annotations

AppointmentBookingService.java

```
@Service
@Slf4j
public class AppointmentBookingService {

    // Combined: TimeLimiter → Retry → CircuitBreaker (applied outer to inner)
    @CircuitBreaker(name = "doctorServiceCB", fallbackMethod = "bookingFallback")
    @Retry(name = "doctorServiceRetry")
    @TimeLimiter(name = "doctorServiceTL")
    public CompletableFuture<AppointmentDTO> bookAppointment(BookingRequest req) {
        return CompletableFuture.supplyAsync(() -> {
            // 1. Get doctor details
            DoctorDTO doctor = doctorClient.getDoctorById(req.getDoctorId());

            // 2. Check availability
            AvailabilityDTO avail = doctorClient.checkAvailability(
                req.getDoctorId(), req.getDate(), req.getTime());

            if (!avail.isAvailable()) {
                // Business exception — does NOT trigger circuit breaker
                throw new SlotNotAvailableException("Slot already booked");
            }

            // 3. Create appointment
            return createAndSaveAppointment(req, doctor);
        });
    }

    // CRITICAL: Same signature as bookAppointment + Throwable as LAST param
    private CompletableFuture<AppointmentDTO> bookingFallback(
            BookingRequest req, Throwable throwable) {

        log.error("Booking fallback for patient={}, doctor={}. Reason: {}",
            req.getPatientId(), req.getDoctorId(), throwable.getMessage());

        // Save PENDING appointment — confirm when service recovers
        Appointment pending = savePendingAppointment(req);
        scheduleRetry(pending.getId());

        return CompletableFuture.completedFuture(AppointmentDTO.builder()
            .status("PENDING_CONFIRMATION")
            .message("Request received. Confirmation within 30 minutes.")
            .confirmationCode(pending.getCode())
            .build());
    }
```

```
}
```

6.6 Common Mistakes

21. **Wrong fallback signature:** The fallback method must have IDENTICAL parameters to the original method, plus a Throwable (or subclass) as the LAST parameter. Missing the Throwable causes Resilience4j to silently skip the fallback and re-throw the exception.
22. **Business exceptions opening the circuit:** DoctorNotFoundException (HTTP 404) is a valid business response. If you include it in record-exceptions, every search for a non-existent doctor opens the circuit — even though Doctor Service is perfectly healthy. Always configure ignore-exceptions for business errors.
23. **Retrying POST requests:** POST /appointments creates a new appointment. If the call succeeds but the response network fails, retrying creates DUPLICATE appointments. Only retry idempotent operations (GET, PUT with the same body).
24. **Circuit flapping:** If minimum-number-of-calls=2 and wait-duration=5s, the circuit opens on 2 failures, waits 5 seconds, goes HALF-OPEN, service still recovering → opens again. Set minimum-number-of-calls=10 and wait-duration=30s for stable behavior.

6.7 Module Summary

Concept	Key Takeaway
CLOSED → OPEN	Failure rate exceeds threshold after minimum-number-of-calls
OPEN state benefit	Fail fast (0ms) instead of waiting for timeout (3-30s)
HALF-OPEN	Probe recovery with limited calls before fully closing
Fallback signature	MUST exactly match original + Throwable as last param
ignore-exceptions	Business errors (404, 409) must NOT open the circuit
Retry + backoff	Exponential backoff prevents thundering herd on recovery
Bulkhead	Isolate thread pools per downstream — contain failures to one service
register-health-indicator=true	CB state visible in /actuator/health for monitoring

MODULE 07

Consul

Service Discovery + K/V Config + Service Mesh by HashiCorp

Chapter 7: HashiCorp Consul

7.1 Why Consul? Eureka Has Served Us Well

Eureka is excellent for Java/Spring-only ecosystems. As systems grow, new requirements emerge that Eureka cannot address:

- MediCare adds a Python ML service for AI-based diagnosis — it cannot use the Eureka Java client
- A Node.js real-time notification service joins the ecosystem
- The compliance team requires strong consistency in service discovery (CP, not AP)
- Operations wants richer health checks — not just heartbeat, but actual DB connectivity checks
- The team wants to eliminate Config Server + RabbitMQ infrastructure for config refresh

Consul addresses all five requirements in a single tool.

7.2 Consul vs Eureka

Aspect	Eureka	Consul
CAP Model	AP (Available)	CP (Consistent)
Language Support	Java/Spring only	Any language (HTTP API)
Health Checks	Heartbeat every 30s	HTTP/TCP/Script/TTL (10s)
Eviction Speed	90 seconds	~30 seconds
Config Storage	No (needs external)	Yes (built-in K/V store)
Service Mesh	No	Yes (Consul Connect)
Multi-Datacenter	Limited	Native support
DNS Discovery	No	Yes (port 8600)
UI Dashboard	Basic	Rich web UI
Refresh Mechanism	Spring Cloud Bus + RabbitMQ	Built-in watch (long poll)

7.3 Setting Up Consul

Running Consul

```
# Install and run Consul (development mode)
consul agent -dev -ui -client=0.0.0.0

# Or with Docker:
docker run -d \
  --name consul-dev \
  -p 8500:8500 \
  -p 8600:8600/udp \
  hashicorp/consul:1.17 \
  agent -server -bootstrap -ui -client=0.0.0.0

# Access Consul UI at: http://localhost:8500
# Services, health checks, K/V config — all visible in the UI

# PRODUCTION: Always run 3 or 5 server nodes (Raft consensus needs quorum)
# 3 nodes: tolerates 1 failure
# 5 nodes: tolerates 2 failures
# NEVER use -dev in production (no persistence — restart loses all data)
```

7.4 Spring Cloud Consul Dependencies

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-consul-discovery</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-consul-config</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

7.5 Configuration — application.properties

application.properties — Consul Full Configuration

```
# ── APPLICATION IDENTITY ──────────────────────────────────────────
spring.application.name=doctor-service
server.port=8081

# ── CONSUL CONNECTION ─────────────────────────────────────────────
spring.cloud.consul.host=localhost
spring.cloud.consul.port=8500

# ── SERVICE DISCOVERY REGISTRATION ────────────────────────────────
spring.cloud.consul.discovery.enabled=true
spring.cloud.consul.discovery.prefer-ip-address=true
spring.cloud.consul.discovery.health-check-path=/actuator/health
spring.cloud.consul.discovery.health-check-interval=10s
# Consul actively calls this endpoint every 10 seconds
spring.cloud.consul.discovery.health-check-timeout=5s
spring.cloud.consul.discovery.health-check-critical-timeout=30s
# AUTO-DEREGISTER if health check fails for 30 consecutive seconds

spring.cloud.consul.discovery.instance-
id=${spring.application.name}:${server.port}:${random.value}
# Unique instance ID — prevents collisions when multiple instances run

# Service metadata (used for routing, monitoring, canary deployments)
```

```
spring.cloud.consul.discovery.metadata.version=2.0.0
spring.cloud.consul.discovery.metadata.zone=us-east-1
spring.cloud.consul.discovery.metadata.team=doctor-platform

# ── CONSUL CONFIG (replaces Config Server) ──────────────────────────
spring.cloud.consul.config.enabled=true
spring.cloud.consul.config.prefix=config
# All config stored under: config/ in the K/V store

spring.cloud.consul.config.default-context=application
# Shared config at: config/application/data

spring.cloud.consul.config.profile-separator=,
# doctor-service + prod profile reads: config/doctor-service,prod/data

spring.cloud.consul.config.format=YAML
# Store config as YAML string in the data key

spring.cloud.consul.config.data-key=data
# The actual key name holding the config content

spring.cloud.consul.config.fail-fast=true

# ── WATCH (Auto Config Refresh) ─────────────────────────────────────
spring.cloud.consul.config.watch.enabled=true
spring.cloud.consul.config.watch.delay=1000
# Long-poll Consul for changes every 1 second.
# When config changes in K/V store, ALL instances refresh in ~1-2 seconds.
# NO RabbitMQ. NO Spring Cloud Bus. Built-in.
```

7.6 Storing Config in Consul K/V

consul CLI — Storing Configuration

```
# Put shared config (all services)
consul kv put config/application/data "
eureka:
  client:
    enabled: false
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
"

# Put doctor-service specific config
consul kv put config/doctor-service/data "
server:
  port: 8081
feature:
  max-appointments-per-day: 20
  online-consultation: true
"

# Put doctor-service PRODUCTION config
consul kv put config/doctor-service,prod/data "
spring:
  datasource:
    url: jdbc:mysql://prod-mysql:3306/medicare_prod
    username: medicare_prod_user
    password: SecureProductionPassword
```

```
feature:
  max-appointments-per-day: 30
"

# Verify: curl http://localhost:8500/v1/kv/config/doctor-service/data?raw
```

7.7 Consul Security — ACLs

application.properties — Consul ACL

```
# In production, always enable ACL (Access Control List)
# Each service gets its own token with minimum required permissions

# application.properties — Doctor Service (with ACL token)
spring.cloud.consul.token=${CONSUL_ACL_TOKEN}
# Token grants Doctor Service permission to:
# - Register/deregister itself
# - Read config/doctor-service/* and config/application/*
# - Cannot read config/billing-service/* (other services private config)

# Without ACL: any service can read ANY other service config
# This is a serious security risk in multi-team organizations
```

7.8 Common Mistakes

25. **Running Consul -dev in production:** Development mode stores everything in memory. Consul restart = ALL service registrations lost, ALL K/V config gone. Always use server mode with a data-dir for persistence.
26. **Not setting health-check-critical-timeout:** Without this, crashed services stay in the Consul registry forever. Always set it so crashed services auto-deregister after ~30 seconds.
27. **Not enabling ACLs in production:** Default Consul has zero authentication. Any service — or attacker who reaches port 8500 — can read all configuration, including database passwords. Enable ACLs and issue per-service tokens.
28. **Using PROPERTIES format instead of YAML in K/V:** With PROPERTIES format, Consul treats each property as a separate K/V entry. This becomes unmanageable at scale. Use YAML format to store the entire config as one blob.

7.9 Module Summary

Concept	Key Takeaway
CP vs AP	Consul = CP (refuses stale data in partitions); Eureka = AP (keeps serving)
Consul Agent	Lightweight process on every machine — distributed health checking
Health Checks	Active HTTP/TCP/Script polls every 10s — detects failures in 30s not 90s
K/V Store	Replace Config Server + Git + Bus with Consul K/V + Watch (1-2s refresh)
Watch mechanism	Long-poll Consul for changes — no RabbitMQ, no Spring Cloud Bus needed

DNS Discovery	service-name.service.consul — any language, not just Java
ACL tokens	Always enable in production — each service has minimum required permissions
health-check-critical-timeout	Always set — auto-deregisters crashed services after N seconds

MODULE 08

Service Mesh

mTLS, Zero-Trust Security, Automatic Observability

Chapter 8: Service Mesh — Istio & Consul Connect

8.1 The Problem — Application-Level Networking at Scale

After seven modules, MediCare has excellent resilience, discovery, and configuration. But all of it lives inside the application code — in Spring Boot dependencies, annotations, and configuration files. As the organization scales to 500 services across 30 teams, several problems emerge that application-level solutions cannot cleanly address:

- **mTLS:** Each of the 500 services needs TLS certificates, certificate rotation logic, and TLS configuration code. 500 services × 3 environments = 1,500 certificate management tasks.
- **Consistent observability:** Distributed tracing requires every developer to correctly add Sleuth, configure Zipkin, and propagate trace headers. One service that forgets breaks the entire trace chain.
- **Canary deployments:** Routing 5% of traffic to a new version requires complex feature flag code and infrastructure support that each team implements differently.
- **Zero-trust security:** Billing Service should not be able to call Patient Service's internal endpoints. Enforcing this without a service mesh requires custom authorization code in every service that might receive unauthorized calls.

A Service Mesh moves ALL of these networking concerns OUT of application code and into the infrastructure layer — a transparent proxy that wraps every service.

8.2 The Sidecar Proxy Pattern

Every service instance gets a sidecar proxy deployed alongside it as a separate process. In Kubernetes, this is a second container in the same Pod. The sidecar intercepts ALL inbound and outbound network traffic using iptables rules injected automatically — the application never knows the sidecar exists.

With vs Without Service Mesh

```
WITHOUT SERVICE MESH:
Doctor Service ─── plain HTTP ──────────────────→ Patient Service
  (Feign client, Resilience4j, LoadBalancer in JAR)
  (TLS config, tracing code in JAR)
  (Application has 10+ Spring Cloud dependencies)

WITH SERVICE MESH (Istio):
Doctor     Envoy         network           Envoy    Patient
Service    Sidecar                         Sidecar  Service
  │          │                               │          │
  ├──HTTP────→│ intercepts outbound           │          │
```

```
   │            ├──mTLS encrypted──────────────→│           │
   │            │ load balanced                 │ decrypt   │
   │            │ retried if needed             ├──HTTP───→│
   │            │ traced automatically          │           │
   │←──HTTP───│←──mTLS encrypted──────────────│           │
   │            │                               │           │

The Doctor Service code is IDENTICAL in both cases.
Zero TLS code. Zero tracing code. Zero load balancing code.
All moved to the Envoy sidecar layer.
```

8.3 Control Plane vs Data Plane

A service mesh has two layers:

- **Data Plane:** The sidecars themselves (Envoy proxies). They handle the actual traffic — TLS, load balancing, retries, circuit breaking, metrics collection. One sidecar per service instance.
- **Control Plane:** Central management system (Istio's istiod). It issues certificates to all sidecars, distributes traffic policies, collects telemetry, and enforces access control. It manages the sidecars without touching application code.

8.4 Installing Istio

Installing Istio

```
# Download Istio CLI
curl -L https://istio.io/downloadIstio | sh -
export PATH=$PWD/istio-1.20.0/bin:$PATH

# Install Istio on Kubernetes (demo profile for development)
istioctl install --set profile=demo -y

# Verify: all Istio components running
kubectl get pods -n istio-system
# istiod-xxx  1/1  Running     ← Control plane
# istio-ingressgateway-xxx  1/1  Running   ← External traffic entry

# Enable AUTOMATIC sidecar injection for medicare namespace
kubectl label namespace medicare istio-injection=enabled

# Now: every Pod deployed to medicare namespace automatically gets
# an Envoy sidecar container injected. No Deployment changes needed.

# Verify a pod has 2 containers (app + sidecar):
kubectl get pods -n medicare
# doctor-service-7d9f97-xkn4p   2/2   Running   ← 2 containers!
```

8.5 application.properties with Istio

application.properties — Doctor Service with Istio

```
# application.properties — Doctor Service running under Istio
```

```
# ── APPLICATION IDENTITY ──────────────────────────────────────────
spring.application.name=doctor-service
server.port=8081

# ── NO EUREKA / CONSUL NEEDED ─────────────────────────────────────
# Kubernetes DNS replaces service discovery
# doctor-service resolves to doctor-service.medicare.svc.cluster.local
eureka.client.enabled=false

# ── NO TLS CONFIGURATION NEEDED ───────────────────────────────────
# Istio sidecar handles mTLS automatically
# Application listens on plain HTTP — sidecar encrypts/decrypts

# ── FEIGN CLIENT ──────────────────────────────────────────────────
# Service names use Kubernetes DNS (not Eureka service names)
spring.cloud.openfeign.client.config.patient-service.url=http://patient-service:8082
# Istio sidecar intercepts, adds mTLS, load balances automatically

# ── TRACING ───────────────────────────────────────────────────────
management.tracing.sampling.probability=0.1
# 10% sampling — Istio sidecar captures ALL requests at network level
# Application only needs to PROPAGATE trace headers (auto in Spring Boot 3)
management.zipkin.tracing.endpoint=http://jaeger-collector:9411/api/v2/spans

# ── METRICS ───────────────────────────────────────────────────────
management.metrics.export.prometheus.enabled=true
management.endpoints.web.exposure.include=health,info,metrics,prometheus
# Istio Prometheus scrapes both /actuator/prometheus AND Envoy sidecar metrics

# ── KEEP RESILIENCE4J FOR APPLICATION-LEVEL PROTECTION ────────────
# Defense in depth: mesh handles network-level, app handles business-level
resilience4j.circuitbreaker.instances.patientServiceCB.sliding-window-size=10
resilience4j.circuitbreaker.instances.patientServiceCB.failure-rate-threshold=50
```

8.6 Enabling Mutual TLS (mTLS)

One Kubernetes resource enforces mTLS across the entire MediCare namespace. Zero application code required:

kubernetes/mtls-policy.yaml — STRICT mTLS

```
# kubernetes/mtls-policy.yaml
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: medicare-mtls-strict
  namespace: medicare
spec:
  mtls:
    mode: STRICT
    # STRICT:      Only mTLS connections. Plain HTTP rejected.
    # PERMISSIVE:  Accept both. Use during migration to service mesh.
    # DISABLE:     No mTLS. Never use in production.

# Apply: kubectl apply -f kubernetes/mtls-policy.yaml

# What Istio does automatically after this:
# 1. Issues X.509 certificate to EVERY service Pod (SPIFFE identity)
# 2. Rotates certificates every 24 hours
# 3. Enforces that only services with valid Istio-issued certs can communicate
# 4. Encrypts ALL traffic between services — HIPAA compliant in-transit encryption
#
# Service Identity (in certificate):
# spiffe://cluster.local/ns/medicare/sa/doctor-service
```

```
# Patient Service KNOWS it is talking to doctor-service — cryptographically.
```

8.7 Canary Deployment with Traffic Splitting

kubernetes/ — Canary Deployment

```
# Deploy v2 alongside v1 (in kubernetes/doctor-service-v2.yaml)
# Both Deployments labeled: app: doctor-service, version: v1 or v2

# DestinationRule — defines the two subsets
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: doctor-service
  namespace: medicare
spec:
  host: doctor-service
  subsets:
    - name: v1
      labels: { version: v1 }
    - name: v2
      labels: { version: v2 }

# VirtualService — 95% to stable v1, 5% to new v2
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: doctor-service
  namespace: medicare
spec:
  hosts: [doctor-service]
  http:
    - route:
        - destination: { host: doctor-service, subset: v1 }
          weight: 95
        - destination: { host: doctor-service, subset: v2 }
          weight: 5

# Gradual rollout:
# kubectl patch virtualservice doctor-service --type merge ...
# Change weights: 90/10 → 50/50 → 0/100
# Instant rollback: change back to 100/0 in one kubectl command
```

8.8 Zero-Trust Authorization

kubernetes/ — Authorization Policies

```
# Default deny — nothing can call anything
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: default-deny-all
  namespace: medicare
spec: {}   # Empty = deny all

# Explicit allow — Appointment Service CAN call Doctor Service
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: doctor-service-access
  namespace: medicare
```

```
spec:
  selector:
    matchLabels: { app: doctor-service }
  rules:
    - from:
        - source:
            principals:
              - "cluster.local/ns/medicare/sa/appointment-service"
      to:
        - operation:
            methods: [GET, POST]
            paths: ["/doctors/*"]

# Result:
# appointment-service → doctor-service: ALLOWED (cryptographically enforced)
# billing-service → doctor-service: DENIED (no authorization policy allows this)
# Any external caller without Istio certificate → DENIED
```

8.9 Automatic Observability

Istio Observability Stack

```
# Install all Istio addons (takes ~60 seconds)
kubectl apply -f samples/addons/prometheus.yaml
kubectl apply -f samples/addons/grafana.yaml
kubectl apply -f samples/addons/jaeger.yaml
kubectl apply -f samples/addons/kiali.yaml

# Open Kiali — real-time service topology
istioctl dashboard kiali
# Shows: live service graph, traffic flow, error rates
# GREEN edges: healthy traffic
# RED edges: high error rate → immediate visual alert

# Open Jaeger — distributed traces
istioctl dashboard jaeger
# Shows: POST /book/appointment → Gateway → Appointment Svc → Doctor Svc
# Exact millisecond breakdown per service — no tracing code in applications

# Open Grafana — metrics dashboards
istioctl dashboard grafana
# Pre-built Istio dashboards: RPS, P99 latency, error rate per service
# Four Golden Signals automatically collected from Envoy sidecars

# All of this — zero application code required.
# Every request traced. Every service measured. Every edge secured.
```

8.10 Consul Connect — Simpler Service Mesh

If your organization is already running Consul (Chapter 7), Consul Connect provides service mesh capabilities without installing Istio:

Consul Connect — Service Mesh

```
# Enable Consul Connect (add to Consul agent configuration)
# consul.hcl
connect {
  enabled = true
}
```

```
# Define service-to-service access with Intentions (simpler than Istio YAML)

# Allow: Appointment Service → Doctor Service
consul intention create appointment-service doctor-service

# Deny: Billing Service → Doctor Service
consul intention create -deny billing-service doctor-service

# List all intentions
consul intention list
# Source                Destination        Action
# appointment-service   doctor-service     allow
# billing-service       doctor-service     deny
# *                     *                  deny    ← default deny all

# Consul Connect uses Envoy proxy (same as Istio) as the sidecar.
# Differences from Istio:
# + Simpler configuration (Intentions vs complex YAML)
# + Works on VMs and bare metal, not just Kubernetes
# + Single tool for discovery + config + mesh
# - Less rich traffic management (no fault injection, traffic mirroring)
# - Less sophisticated observability UI
```

8.11 Common Mistakes

29. **Not planning for Envoy resource overhead:** Each Envoy sidecar uses ~50-100MB RAM and adds ~1-2ms latency per hop. For 50 services × 3 pods each = 150 sidecars × 75MB = ~11GB additional cluster RAM. Plan capacity before enabling the mesh.
30. **Enabling STRICT mTLS before all services have sidecars:** If some services have sidecars and some do not, STRICT mode breaks communication between them. Always use PERMISSIVE mode during migration, then switch to STRICT once all services have sidecars.
31. **Removing application-level resilience:** Some teams remove Resilience4j when they add Istio, thinking the mesh handles everything. Wrong. Mesh retries handle network failures. Resilience4j handles business logic failures. Keep both — defense in depth.
32. **Forgetting trace header propagation in async code:** Istio automatically propagates B3 trace headers in synchronous HTTP calls. But async code (CompletableFuture, @Async methods) loses the trace context. Manually propagate X-B3-TraceId, X-B3-SpanId, X-B3-Sampled in async boundaries.

8.12 Module Summary

Concept	Key Takeaway
Service Mesh purpose	Move ALL networking concerns (TLS, tracing, LB) OUT of application code
Sidecar proxy	Envoy runs alongside every service — intercepts all traffic transparently
Control Plane (istiod)	Manages all sidecars — issues certs, distributes policies, collects telemetry
mTLS with PeerAuthentication	One YAML file → every service-to-service connection encrypted + authenticated
STRICT mode migration	Start PERMISSIVE (accept all) → add sidecars → switch to STRICT
VirtualService + weights	Canary deployments — one kubectl patch changes traffic split instantly

AuthorizationPolicy	Zero-trust: default deny all, explicitly allow each communication path
Kiali + Jaeger + Grafana	Full observability in 60 seconds — zero application instrumentation
Consul Connect	Simpler mesh — best when already running Consul, or need VM support

Complete Masterclass Summary

This table captures the single most important production rule from each module — the insight that separates developers who build systems that work from those who build systems that almost work.

#	Module	Most Critical Production Rule
01	**Eureka**	prefer-ip-address=true always. In Docker/K8s, container hostnames are not resolvable across containers. Without this, all inter-service calls fail.
02	**Gateway**	Never import spring-boot-starter-web. Gateway uses WebFlux (reactive). The web starter causes an irreconcilable bean conflict and fails at startup.
03	**Feign**	@PathVariable MUST have an explicit name in Feign interfaces. @PathVariable("id") not @PathVariable. Without it, Feign fails at startup — silently in some versions.
04	**Load Balancer**	Never retry POST requests. A POST that creates a resource and retries on response failure creates DUPLICATE records — duplicate appointments, payments, or users.
05	**Config Server**	Config Server cannot use Config Server for its own config. Its application.properties must be local. Circular dependency = startup deadlock.
06	**Resilience4j**	ignore-exceptions must include business errors. DoctorNotFoundException opening the circuit on every 404 closes the system to every booking attempt.
07	**Consul**	Never use consul agent -dev in production. Dev mode has zero persistence. Consul restart = all registrations and all K/V config deleted permanently.
08	**Service Mesh**	Use PERMISSIVE mTLS mode during migration. Switching to STRICT before all services have sidecars immediately breaks all communication between mesh and non-mesh services.

Learning Path Recommendation

These modules build on each other. The recommended study order for maximum retention:

33. Start with Eureka (Module 1) — service discovery is the foundation everything else builds on.
34. Add Gateway (Module 2) — now you have a complete ingress layer.
35. Add Feign + LoadBalancer (Modules 3-4) — your services can communicate efficiently.
36. Add Config Server (Module 5) — externalize configuration before you have dozens of services.
37. Add Resilience4j (Module 6) — build for failure, not for success.
38. Evaluate Consul (Module 7) — migrate from Eureka when you need polyglot or stronger consistency.
39. Consider Service Mesh (Module 8) — adopt when application-level networking becomes a maintenance burden across teams.

TIP: Modules 1-6 are the production baseline for any Spring Boot microservices system.
Modules 7-8 are architecture-level decisions that depend on organizational scale and requirements.
Build the baseline first. Layer in complexity only when you genuinely need it.